052400-1 VU Information Management and Systems Engineering (2024W)

Milestone 1

Group 2

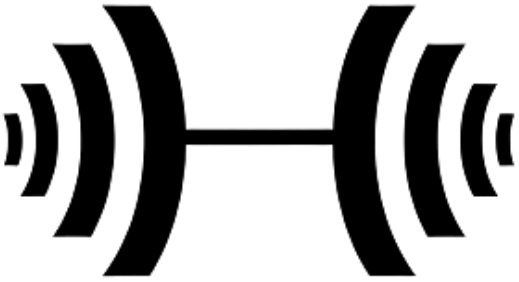
Student 1: Bratanov, Ivan, 12228892

Student 2: Yordanov, Georgi, matriculation number

October 22, 2024, Vienna

Contents

Huge



[Logo generated by Imagen 3 (Gemini)]

“Huge” is the name of a small (fictional) gym chain which requires a simple database to manage the data it uses. In the following document, you will find a detailed description of the design of this system.

Milestone 1

1.1 Conceptual Modeling

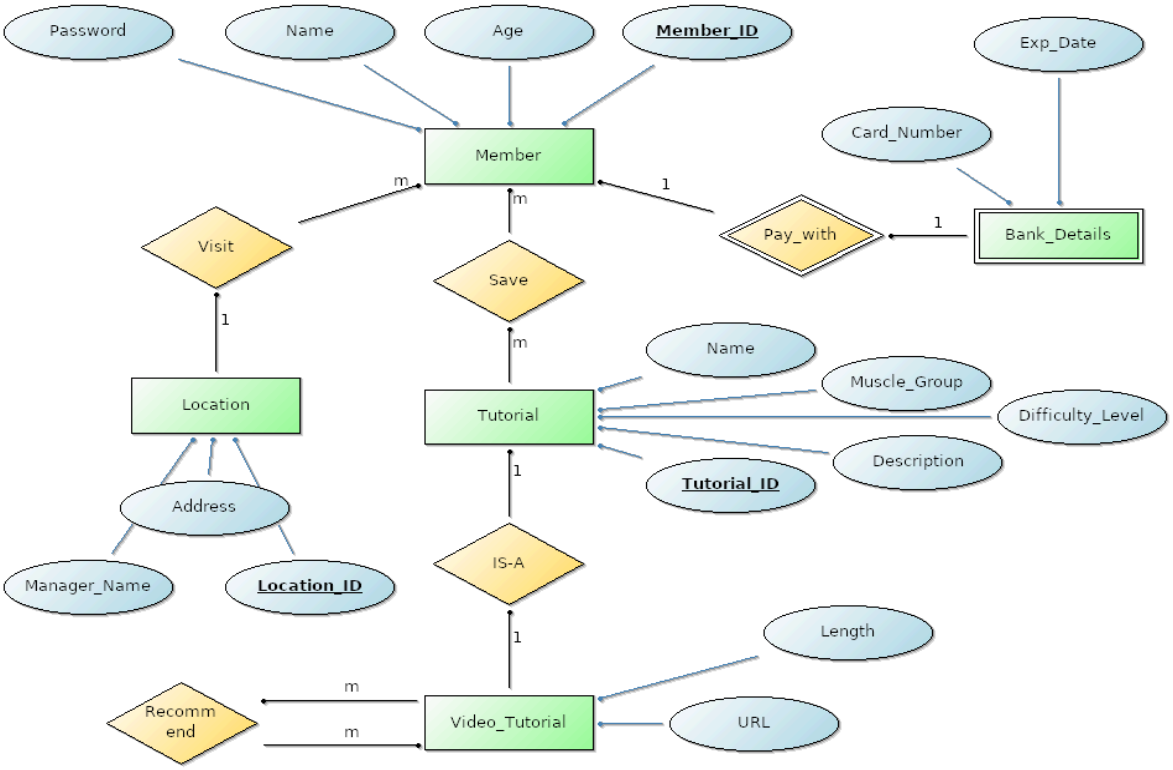
1.1.1 Application Domain

- The gym database manages Members which have a Name, Age and a Password. We make sure all of our members are at least 16 years old.
- Each member uses exactly one bank card to pay for our service, and we save the Bank Details associated with them. We save the Card Number and the Expiration Date, but not the CVC.
- We have multiple Locations, each one of them having an Address and the Name of the manager (we are a small gym chain and have only a few managers, thus further details are not required). Each Manager is responsible for only one gym, and each gym has only one manager.
- Each one of our Members can visit only one of our Locations at a time. But (of course) each location is visited by many members.
- Each Member can save many of our Tutorials to their account (and each Tutorial, of course, can be saved by many Members)
- The Tutorials have a Difficulty level, Target muscle group, and a text description of the exercise.

- Some of our Tutorials are Video Tutorials. In addition to all the things the regular Tutorials have, these contain the Length and the URL of the video.
- Video Tutorials can recommend any number of other Video Tutorials and every Video Tutorial can be recommended by many Video Tutorials.
- Every Entity has a unique ID

Logical Design – ER Diagram

Diagram created with BeeUp



Relational Modeling – SQL CREATE Statements

-> BeeUp ERD generated code was used as a template for these sql create statements

```
-- Will use MySQL
```

```
-- Relations
```

```
-- @block

CREATE TABLE saved (

  member_id INT NOT NULL,

  tutorial_id INT NOT NULL,

  CONSTRAINT pk_save PRIMARY KEY (member_id, tutorial_id)

);







-- @block

CREATE TABLE recommend (

        recommender_tutorial_id INT NOT NULL,

        recommended_tutorial_id INT NOT NULL,

        CONSTRAINT pk_recommend PRIMARY KEY (recommender_tutorial_id,
recommended_tutorial_id)

);







-- Entities



-- @block

CREATE TABLE member (

  member_id INT AUTO_INCREMENT,

  name VARCHAR(32),

  age INT CHECK(age > 16),

  password VARCHAR(32),
```

```
  location_id INT,
  CONSTRAINT pk_member PRIMARY KEY (member_id)
);

-- @block
CREATE TABLE bank_details (
  member_id INT NOT NULL,
  card_number INT,
  exp_date VARCHAR(32),
  CONSTRAINT pk_bank_details PRIMARY KEY (member_id),
  FOREIGN KEY (member_id) REFERENCES member(member_id)
);

-- @block
CREATE TABLE location (
  location_id INT NOT NULL,
  manager_name VARCHAR(32),
  address VARCHAR(32),
  CONSTRAINT pk_location PRIMARY KEY (location_id)
);

-- @block
CREATE TABLE tutorial (
  tutorial_id INT AUTO_INCREMENT,
```

```
  name VARCHAR(32) NOT NULL,

  muscle_group VARCHAR(32),

  difficulty_level VARCHAR(32) CHECK (difficulty_level IN ('beginner', 'intermediate',
'advanced')),

  description VARCHAR(2048),

  CONSTRAINT pk_tutorial PRIMARY KEY (tutorial_id)

);




-- @block

CREATE TABLE video_tutorial (

  tutorial_id INT NOT NULL,

  duration INT,

  url VARCHAR(32),

  CONSTRAINT pk_video_tutorial PRIMARY KEY (tutorial_id)

);







-- Adding the foreign keys



-- @block

-- For the visit 1:m binary relation (adding the location pk to the member variables)

ALTER TABLE member ADD CONSTRAINT fk_visit_location FOREIGN KEY (location_id) REFERENCES
location(location_id);



-- @block
```

```
-- For the save m:m binary relation

ALTER TABLE saved ADD CONSTRAINT fk_save_tutorial FOREIGN KEY (tutorial_id) REFERENCES
tutorial(tutorial_id);




-- @block

ALTER TABLE saved ADD CONSTRAINT fk_save_member FOREIGN KEY (member_id) REFERENCES
member(member_id);




-- @block

-- For the recommend unary relation

ALTER TABLE recommend ADD CONSTRAINT fk_recommender_video FOREIGN KEY
(recommender_tutorial_id) REFERENCES video_tutorial(tutorial_id);




-- @block

ALTER TABLE recommend ADD CONSTRAINT fk_recommended_video FOREIGN KEY
(recommended_tutorial_id) REFERENCES video_tutorial(tutorial_id);




-- @block

ALTER TABLE video_tutorial ADD CONSTRAINT fk_video_tutorial FOREIGN KEY (tutorial_id)
REFERENCES tutorial(tutorial_id);
```

Bratanov Ivan 12228892

Use Case Definition and Design

Use case involving at least 3 entities, two of which are in an **IS-A** relationship:
- member (saves) tutorial (IS-A) video_tutorial

Textual Description

Name: User browses video tutorials and saves one to their member account
Actors: User (Primary), Front-End, Back-End, Database
Preconditions: All actors exist, All the required database tables exist and are filled. User is logged with their member account. The “Tutorials” page is displayed on the front-end. The user decides they want to look through the available video tutorials and save one for later viewing
Postconditions: The chosen video-tutorial is now “saved” to the user’s member account.
Main Flow:

- User selects the “Show video tutorials only” on the “Tutorials” page they are on;
- Front end calls the back-end API.
- Backend filters out the video tutorials out of the database and sends a set of video tutorials to the front-end
- Front end displays the video tutorials
- User browses the video tutorials and finds one that they like;
- They select the item and click on “save to profile”;
- The front-end calls the back-end API. It gives the member_id and the tutorial_id;
- The back-end inserts the member_id and tutorial_id in the “saved”-table of the database, if they are not already there;
- If the insertion is successful, the back-end sends confirmation to the front-end;
- The front-end displays a “success” message.

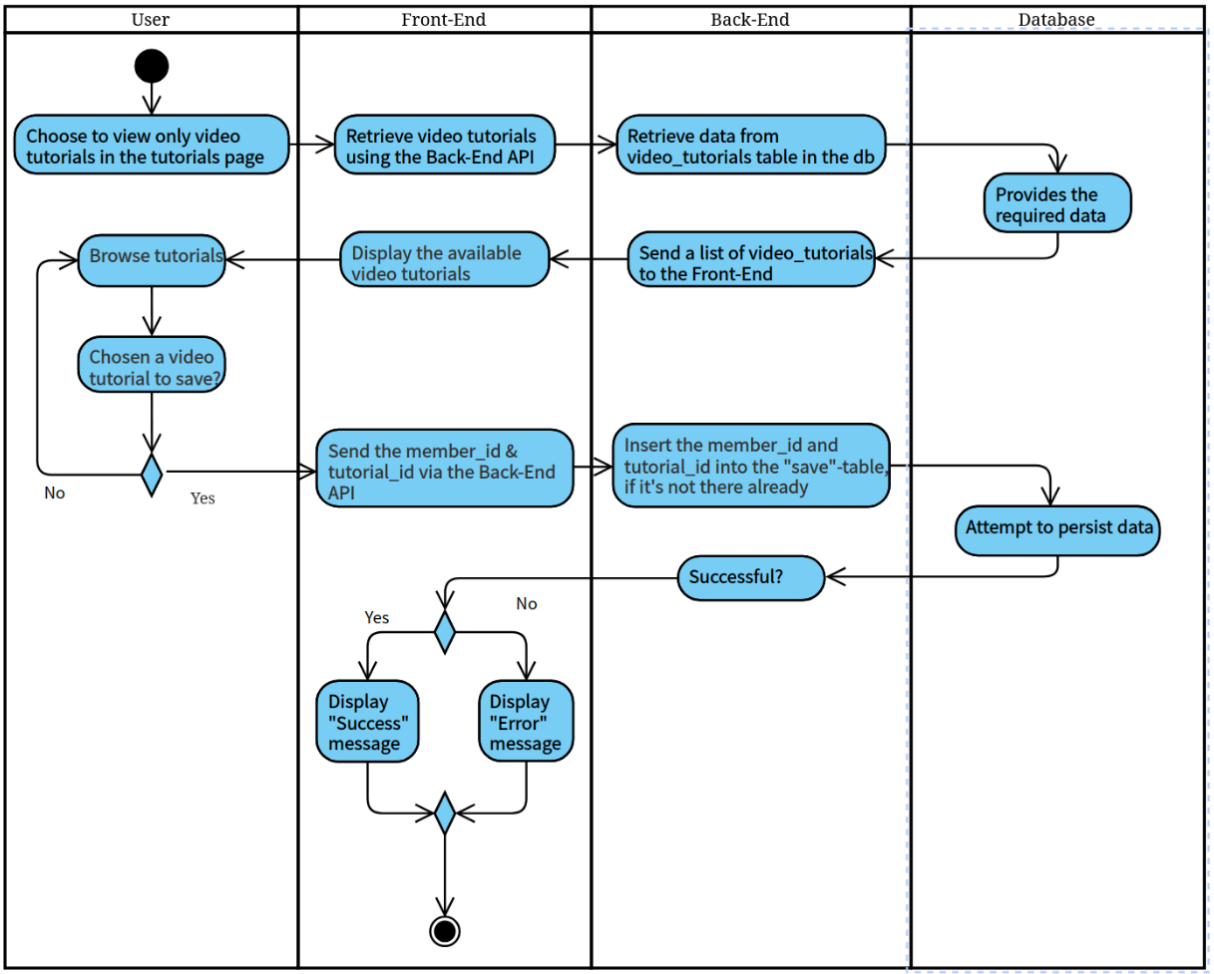
Extension:

- Video tutorial could not be saved;
- Back-end notifies the front-end, that the insertion failed.
- Front-end displays a message to the user, telling them that the item is already saved.

Trigger: User clicking “Video tutorials only” on the front end “Tutorials” page

Graphical Representation

-> Diagram created with visual-paradigm community edition



Bratanov Ivan 12228892

Reporting

- Which beginner video tutorial has been saved the most? What are its name, url, what is the age range of the members who have saved it and how many times has it been saved?
- Will be used to see what is the most popular beginner-friendly video that we have, as well as the rough demographic of the people interested in it
- The data involved in my use case is the saved video tutorial, which will become the report's result if it is saved enough times.
- The 3 non-bridging entities involved are: “member”, “tutorial” and “video_tutorial”. There is also a bridging entity: “saved”
- The report includes data from:
    - “member”: age (min age and max age)
    - “tutorail”: name
    - “video_tutorial”: URL
- The result is filtered by the tutorial “difficulty_level” = “beginner”

Query:

```
SELECT
       t.name AS tutorial_name,
       vt.url,
       MIN(m.age) AS min_age,
       MAX(m.age) AS max_age,
       COUNT(s.tutorial_id) AS times_saved
FROM
       member m
JOIN
       saved s ON m.member_id = s.member_id
JOIN
       tutorial t ON s.tutorial_id = t.tutorial_id
JOIN
       video_tutorial vt ON t.tutorial_id = vt.tutorial_id
WHERE
       t.difficulty_level = 'beginner'
GROUP BY
       t.name, vt.url
ORDER BY
       times_saved DESC
LIMIT 1;
```

Bratanov Ivan 12228892

NoSQL Design

1. Overall Design:

I would group the data in the following way:

- Location, with all of its sql attributes and a nested array of visitors (member_id of people who visit that location)
- Member, with all of its sql attributes and a nested object “bank_details” and an array of “saved_tutorials”
- Tutorial, with all of its sql attributes and with an optional element “video_details” that will contain the video_tutorial attributes

Grouping data in this way will simplify the database and increase performance, since data that logically belongs together will be retrieved more efficiently. On the other hand JOINS become more difficult.

2. Member and Saved

- Since it would often be accessed together, it makes sense to group the data logically connected to a specific member into one document. In this case saved tutorials and bank details could be nested in the member document. This way we can leverage the performance benefits of NoSQL databases.On the other hand, storing the location that each member is associated with would not make sense, since it will introduce a lot of redundancy.

- Schema:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "description": "The data unique to a member account",
  "type": "object",
  "properties": {
            "_id": {
            "type": "string"
             }
            "member_id": {
            "type": "integer"
            },
            "name": {
            "type": "string"
            },
            "age": {
            "type": "integer"
            },
            "saved_tutorials": {
            "type": "array",
            "items": { "type": "integer" }
            },
            "bank_details": {
            "type": "object",
            "properties": {
            "card_number": {
            "type": "string"
            },
            "exp_date": {
            "type": "string",
            "format": "date"
            }
            },
            "required": ["card_number", "exp_date"]
            }
  },
  "required": ["member_id", "name", "age", "saved_tutorials", "bank_details"]
}
```

- Example JSON for member:

```
{
	"_id": "<<some object id>>",
	"member_id": 13,
	"name": "Bratan Ivanov",
	"age": 22,
	"saved_tutorials": [2, 4, 12, 13],
	"bank_details": {
		"card_number": "<<some card number>>",
		"exp_date": "2028-11-01" }
}
```

-> validated with jsoneditoronline.org

3. Tutorial and Video Tutorial

- It makes sense to use the same collection structure for both regular tutorials and video tutorials. We use optional embedding to add the extra data required by video tutorials only where necessary. This way the tutorials are as light-weight as possible, while still containing all relevant data.
- Schema:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
	"_id": {
	"type": "string"
	},
	"tutorial_id": {
	"type": "integer"
	},
	"name": {
	"type": "string"
	},
	"muscle_group": {
	"type": "string"
	},
	"difficulty_level": {
	"type": "string",
	"enum": ["beginner", "intermediate", "advanced"]
	},
	"description": {
	"type": "string"
	},
	"video_details": {
	"type": "object",
	"properties": {
	"url": {
	"type": "string",
	"format": "uri"
	},
	"duration": {
	"type": "integer"
	}
	},
	"required": ["url", "duration"]
	}
  },
  "required": ["_id", "tutorial_id", "name", "muscle_group", "difficulty_level", "description"]
}
```

- Example JSON for tutorial / video tutorial:

```
{
	"_id": "<<some object id>>",
	"tutorial_id": 12,
	"name": "Bench Press",
	"muscle_group": "Chest",
	"difficulty_level": "beginner",
	"description": "<<Some example description>>",
	"video_details": {
		"url": "https://example.com/",
		"duration": 3}
}
```

-> validated with jsoneditoronline.org

4. Use Case

- My use case has 2 parts: retrieving all of the video tutorials (so that they can be presented to the user) and saving one of the tutorials to the user member account.
- In a NoSQL database retrieving all the available tutorials will be trivial:

```
db.tutorials.find({
        "video_details": { $exists: true }
})
```

- Saving a tutorial to a member account is not much more difficult than the MariaDB version. It still requires only the member_id of the account owner and the tutorial_id of the video:

```
db.members.updateOne(
        { member_id: 10 },  // This is the id of the member who is logged in and saving the vt
        {
                $push: {
                        saved_tutorials: 14 // This is the tutorial_id of the selected vt
                }
        }
)
```

Bratanov Ivan 12228892

Sources and Materials

- https://json-schema.org/learn/json-schema-examples
- https://www.youtube.com/watch?v=tE4EbSE65qM
- https://www.youtube.com/watch?v=-bt_y4Loofg
- https://www.mongodb.com/docs/manual/tutorial/
- https://www.youtube.com/watch?v=Cz3WcZLRaWc&t=425s
- https://jsoneditoronline.org/indepth/validate/json-schema-validator/
- https://online.visual-paradigm.com/
- https://www.youtube.com/watch?v=a3v4-KmDbQA&t=138s
- https://chatgpt.com Used for auto generating sql data based on the sql-create statements, correcting text structure and grammar, technical questions
- https://git01lab.cs.univie.ac.at (versioning)
- Document created with https://docs.google.com

- yt tutorials used for example data:
    - https://www.youtube.com/watch?v=vngli9UR6Hw
    - https://www.youtube.com/watch?v=gRVjAtPip0Y
    - https://www.youtube.com/watch?v=EdtaJRBqwes
    - https://www.youtube.com/watch?v=Sf24_x-Godo&t=307s

Yordanov Georgi 12036828

1.1 Yordanov Georgi: Use Case Definition and Design

1.1.1 Textual Description

Name: User wants to buy a membership and for that purpose he gives his bank accounts and then buys a membership
Actors: User (Primary), Front-End, Back-End, Database
Preconditions: All databases exist and the frontend for creation of application and the backend for the modification of the database are working correctly. The user wants to be a member of our fitness. For this purpose, he has to create an account in our database.
Postconditions: The user has paid successfully using his bank details and his membership is created and stored in the database.
Main Flow:

- The user selects the “Create membership” on the main page of the fitness website.
- The front end calls the back-end API.
- Back end gives the template, which the user has to fill, to the front end, which presents it on the screen to the user.
- The user provides all needed information, such as name, age and password and then he clicks on the “PAY” button.
- Then the user receives another template, where he has to fill in his bank details.
- Then he clicks on the button “Finish” to finish the paying process. The front end asks him again, if he is sure that he wants to pay.
- If the user clicks on “Yes” the back end processes the application of the user, such as it takes the money from the bank account of the user and inserts a new record in table “Member” for his membership.
- Also the backend inserts a new record for the bank details of the user in the table “Bank_Details” in the database.
- If the process of buying membership is successful, the user receives all privileges and rights, which this membership can provide.

Extension:

- The user could not have enough money in his bank account to pay for the membership.
- If the process of transfer of money from the user bank account to the fitness studio’s bank account fails, the user receives a message from the front end “Payment failed!”.
- If the payment is successful, the user receives a message from the front end “Payment successful!” and receives all rights, which the membership provides.
- When the contract for the membership expires the user can extend the contract with one click of the button “Extend contract”. The back end automatically uses the bank details of the user, which are stored in the table “Bank_Details”.

Trigger: The user clicks on the button “Create membership” on the main page of the fitness website.

1.1.2 Graphical Representation

1.2 Student 2: Reporting

1.3 Student 2: NoSQL Design